

Matplotlib

Data Visualization Notes

Foundations of Data Science
Comprehensive reference – code, output & insights

Kabir Roy
Registration No.: 23BCE10815
B.Tech CSE, VIT Bhopal University

Contents

1. Introduction to Matplotlib	3
1.1 Real-World Example: Plotting Runs Over Years	3
1.2 Adding Labels and Titles	4
1.3 Multiple Lines with Legends	4
1.4 Markers, Colors & Line Styles	6
1.5 Styles & Themes	7
1.6 XKCD Hand-Drawn Style	7
2. Bar Plots	9
2.1 Basic Vertical Bar Chart	9
2.2 Side-by-Side (Grouped) Bar Chart	9
2.3 Horizontal Bar Chart	10
2.4 Bar Charts with Value Labels	11
2.5 Practice: Multi-Year Grouped Comparison	12
3. Histograms	14
3.1 Basic Histogram	14
3.2 Custom Bin Edges	14
3.3 Adding a Vertical Reference Line: <code>axvline</code>	15
4. Pie Charts	17
4.1 Basic Pie Chart	17
4.2 Custom Colors	17
4.3 Wedge Properties	18
4.4 Exploded Slices	18
4.5 Shadow Effect	19
4.6 Start Angle	20
4.7 Percentage Labels: <code>autopct</code>	20
4.8 When NOT to Use a Pie Chart	21
4.9 Summary of Useful Pie Chart Parameters	22
5. Scatter Plots	23
5.1 Basic Scatter Plot	23
5.2 Size and Color Encoding	23
5.3 Colormaps with NumPy Arrays	24
5.4 Point Annotations	24
5.5 Comparing Two Groups	25
6. Stack Plots	27
6.1 Recap: Pie Chart for a Single Snapshot	27
6.2 What is a Stack Plot?	27
6.3 Weekly Activity Tracker	27
6.4 Key Parameters in <code>stackplot()</code>	28

7. Subplots	30
7.1 <code>plt.subplot()</code> : Row, Column, Index	30
7.2 A 2×2 Grid of Plots	31
7.3 <code>plt.subplots()</code> : The Object-Oriented Way	31
7.4 A 2×2 Grid with the OO Interface	32

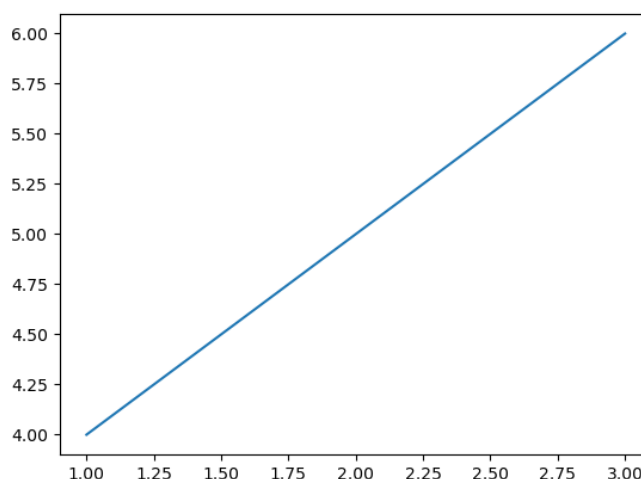
1. Introduction to Matplotlib

Matplotlib is the foundational plotting library in Python's data science stack. It provides a flexible, low-level interface (`pyplot`) to create static, animated, and interactive visualizations.

Basic Plot

```
1 import matplotlib.pyplot as plt
```

```
1 plt.plot([1,2,3],[4,5,6])
```



```
1 plt.show()
```

Interacting with the Plot:

When a plot window appears, you can:

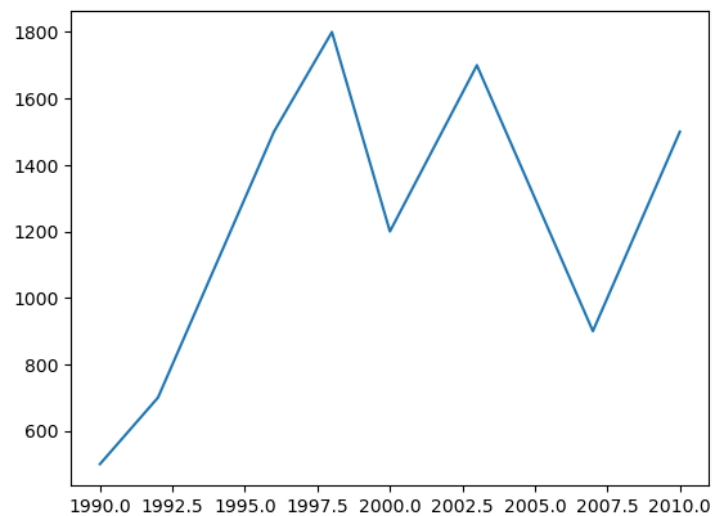
- Zoom in / out
- Pan around
- Use arrows to navigate plot history
- Reset to home view
- Save as PNG using the disk icon

These features are automatically included in the plot window.

Real-World Example: Plotting Runs Over Years

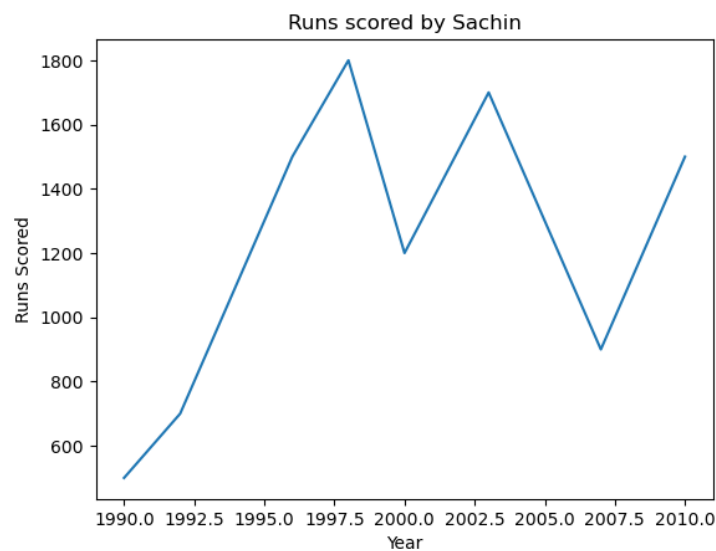
```
1 years = [1990, 1992, 1994, 1996, 1998, 2000, 2003, 2005, 2007, 2010]
2 runs = [500, 700, 1100, 1500, 1800, 1200, 1700, 1300, 900, 1500]
3
```

```
4 plt.plot(years, runs)
5 plt.show()
```



Adding Labels and Titles

```
1 plt.plot(years, runs)
2 plt.xlabel("Year")
3 plt.ylabel("Runs Scored")
4 plt.title("Runs scored by Sachin")
```



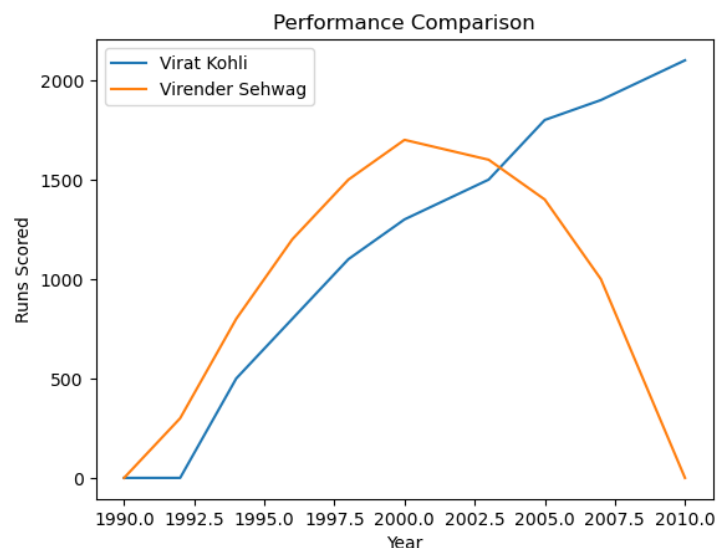
Multiple Lines with Legends

```
1 kohli = [0, 0, 500, 800, 1100, 1300, 1500, 1800, 1900, 2100]
2 sehwa = [0, 300, 800, 1200, 1500, 1700, 1600, 1400, 1000, 0]
3
4 plt.plot(years, kohli, label="Virat Kohli")
5 plt.plot(years, sehwa, label="Virender Sehwag")
```

```

6
7 plt.xlabel("Year")
8 plt.ylabel("Runs Scored")
9 plt.title("Performance Comparison")
10 plt.legend()    # When label is set in plt.plot(), plt.legend() alone is
                  # enough
11 plt.show()

```



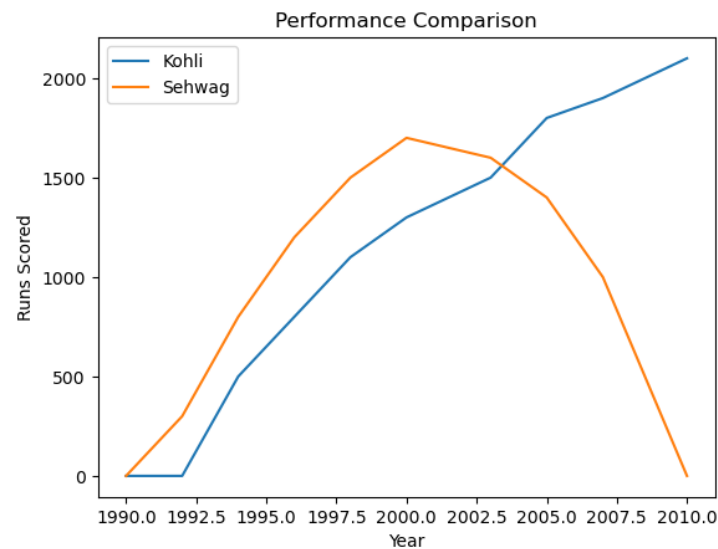
Bad Practice:

Passing a list of names directly to `plt.legend([])` works, but it is fragile – the order must exactly match the plotting order. Prefer the `label=` argument inside each `plt.plot()` call instead.

```

1 plt.plot(years, kohli)
2 plt.plot(years, sehwan)
3 plt.legend(["Kohli", "Sehwag"])
4 plt.xlabel("Year")
5 plt.ylabel("Runs Scored")
6 plt.title("Performance Comparison")
7 plt.show()

```

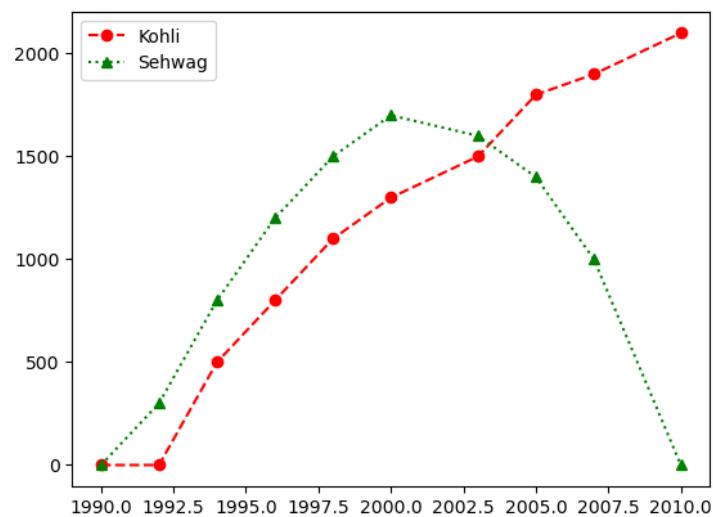


Markers, Colors & Line Styles

```

1 plt.plot(years, kohli, 'ro--', label="Kohli")      # red circles, dashed
  line
2 plt.plot(years, sehwa, 'g^:', label="Sehwag")     # green triangles,
  dotted line
3 plt.legend()
4 % Full marker/linestyle reference: matplotlib.org -> pyplot.plot

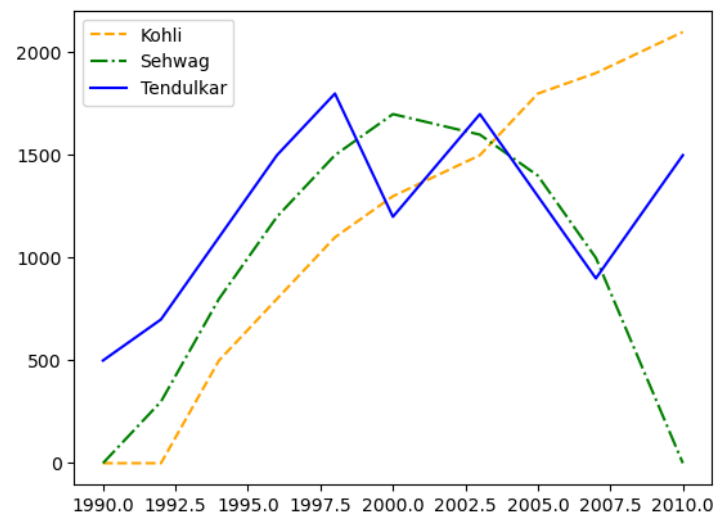
```



```

1 plt.plot(years, kohli, color='orange', linestyle='--', label="Kohli")
2 plt.plot(years, sehwa, color='green', linestyle='-.', label="Sehwag")
3 plt.plot(years, runs, color='blue', label="Tendulkar")
4 plt.legend()

```

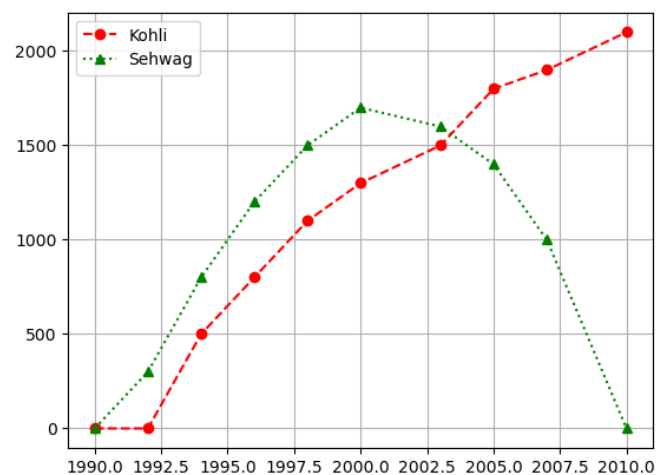


Styles & Themes

```

1 plt.style.available
2
3 plt.plot(years, kohli, 'ro--', label="Kohli")
4 plt.plot(years, sehwa, 'g^:', label="Sehwag")
5 plt.legend()
6 plt.grid(True)
7 plt.style.use("ggplot")

```

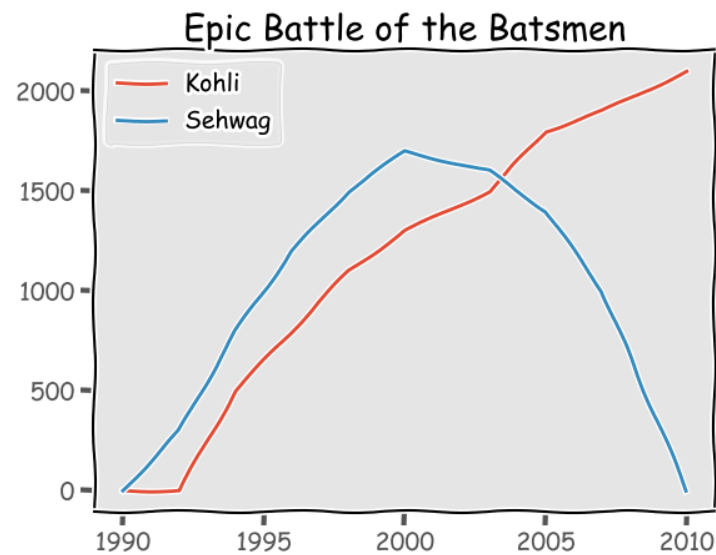


XKCD Hand-Drawn Style

```

1 with plt.xkcd():
2     plt.plot(years, kohli, label="Kohli")
3     plt.plot(years, sehwa, label="Sehwag")
4     plt.title("Epic Battle of the Batsmen")
5     plt.legend()
6     plt.show()

```

**Caution:**

Plotting too many series on one figure (e.g. 50 random lines) overwhelms the viewer and defeats the purpose of visualization – always prefer clarity over density.

```
1 import numpy as np
2
3 for i in range(50):
4     plt.plot(np.random.rand(100), linewidth=1)
5
6 plt.title("Too Much Data Can Be Confusing!")
7 plt.grid(True)
8 plt.tight_layout()
9 plt.show()
```



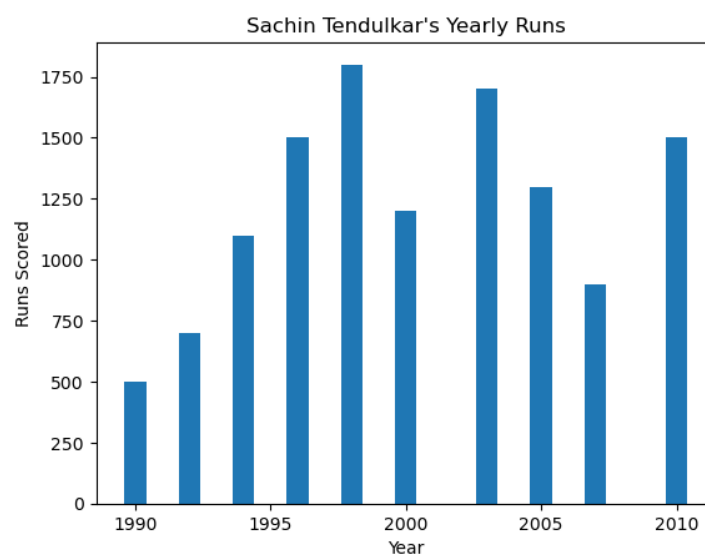
2. Bar Plots

Bar charts are used to **compare quantities** across categories. They are easy to read and powerful for visual analysis.

Basic Vertical Bar Chart

```
1 import matplotlib.pyplot as plt
2
3 years = [1990, 1992, 1994, 1996, 1998, 2000, 2003, 2005, 2007, 2010]
4 runs = [500, 700, 1100, 1500, 1800, 1200, 1700, 1300, 900, 1500]
```

```
1 plt.bar(years, runs)
2 plt.xlabel("Year")
3 plt.ylabel("Runs Scored")
4 plt.title("Sachin Tendulkar's Yearly Runs")
5 plt.show()
```



Side-by-Side (Grouped) Bar Chart

Technique:

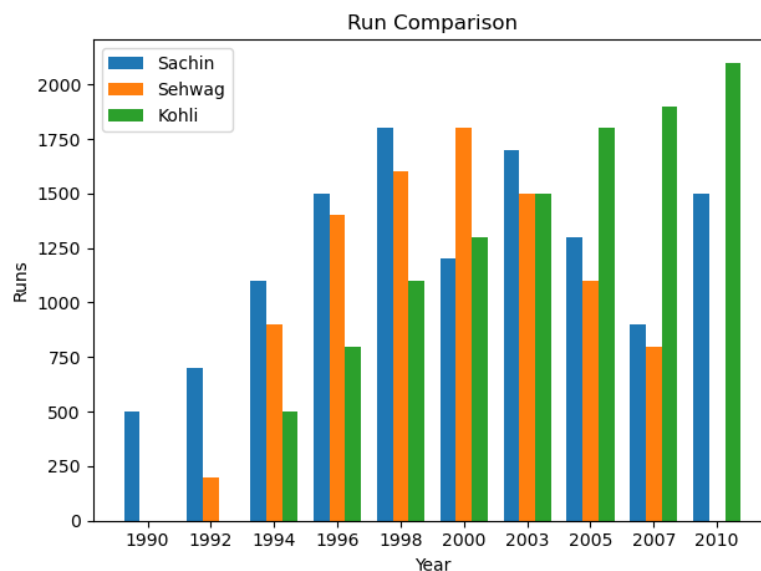
Use `numpy.arange()` to generate index positions for each category, then shift each series left/right by the bar width to place bars side by side.

```
1 import numpy as np
2
3 sachin = [500, 700, 1100, 1500, 1800, 1200, 1700, 1300, 900, 1500]
```

```

4 sehwag = [0, 200, 900, 1400, 1600, 1800, 1500, 1100, 800, 0]
5 kohli = [0, 0, 500, 800, 1100, 1300, 1500, 1800, 1900, 2100]
6
7 x = np.arange(len(years))    # index positions
8 width = 0.25
9
10 plt.bar(x - width, sachin, width=width, label="Sachin")
11 plt.bar(x, sehwag, width=width, label="Sehwag")
12 plt.bar(x + width, kohli, width=width, label="Kohli")
13
14 plt.xlabel("Year")
15 plt.ylabel("Runs")
16 plt.title("Run Comparison")
17 plt.xticks(x, years)        # Show actual year instead of 0,1,2,...
18 plt.legend()
19 plt.tight_layout()
20 plt.show()

```

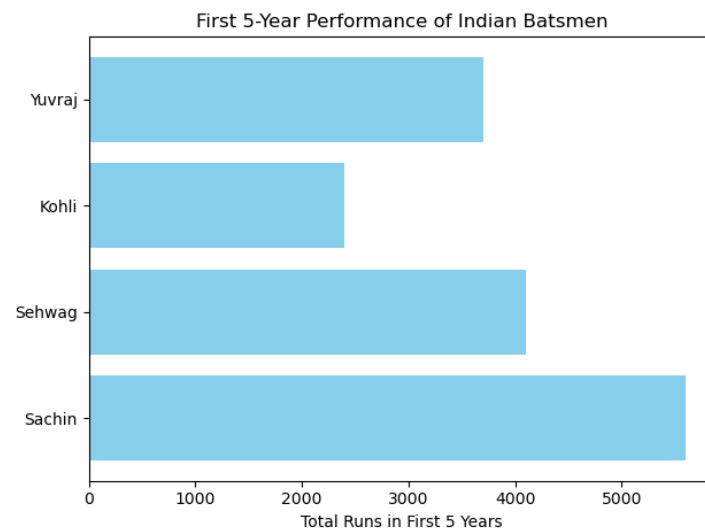


Horizontal Bar Chart

```

1 players = ["Sachin", "Sehwag", "Kohli", "Yuvraj"]
2 runs_5yrs = [500+700+1100+1500+1800, 0+200+900+1400+1600,
3              0+0+500+800+1100, 300+600+800+1100+900]
4
5 plt.barh(players, runs_5yrs, color="skyblue")
6 plt.xlabel("Total Runs in First 5 Years")
7 plt.title("First 5-Year Performance of Indian Batsmen")
8 plt.tight_layout()
9 plt.show()

```



Why Switch x and y?:

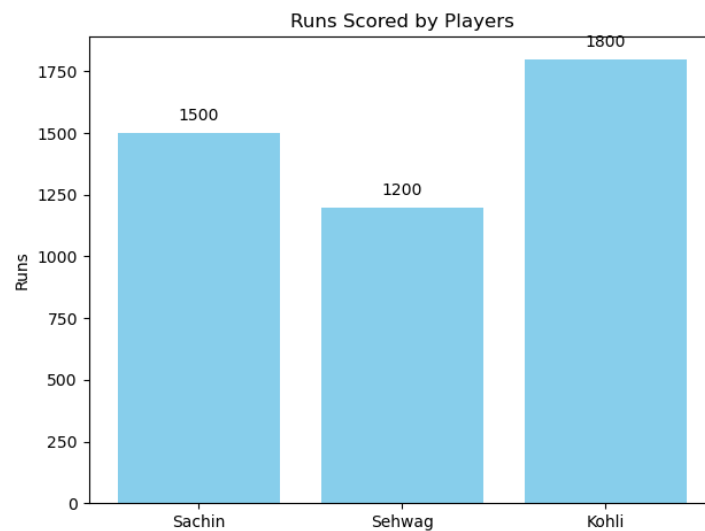
In `bar()`: x = categories, y = values.

In `barh()`: y = categories, x = values.

Horizontal bars help when category names are long, or when comparisons read better left-to-right.

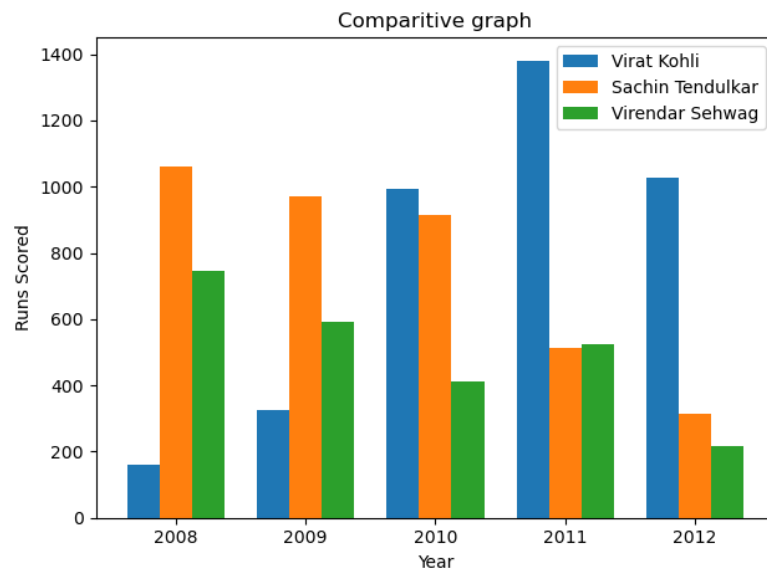
Bar Charts with Value Labels

```
1 import matplotlib.pyplot as plt
2
3 players = ["Sachin", "Sehwag", "Kohli"]
4 runs = [1500, 1200, 1800]
5
6 plt.bar(players, runs, color="skyblue")
7
8 # Add labels on top of bars
9 for i in range(len(players)):
10     plt.text(i, runs[i] + 50, str(runs[i]), ha='center')
11
12 plt.ylabel("Runs")
13 plt.title("Runs Scored by Players")
14 plt.tight_layout()
15 plt.show()
```



Practice: Multi-Year Grouped Comparison

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 years = [2008, 2009, 2010, 2011, 2012]
5 virat = [159, 325, 995, 1381, 1026]
6 sachin = [1060, 972, 914, 513, 315]
7 sehwag = [744, 593, 410, 524, 216]
8
9 x = np.arange(0, 5, 1)
10 width = 0.25
11 plt.bar(x-0.25, virat, width=width, linewidth=1, label="Virat Kohli")
12 plt.bar(x, sachin, width=width, linewidth=1, label="Sachin Tendulkar")
13 plt.bar(x+0.25, sehwag, width=width, linewidth=1, label="Virendar Sehwag")
14
15 plt.xticks(x, years)
16 plt.xlabel("Year")
17 plt.ylabel("Runs Scored")
18 plt.title("Comparitive graph")
19 plt.tight_layout()
20 plt.legend()
21 plt.show()
```



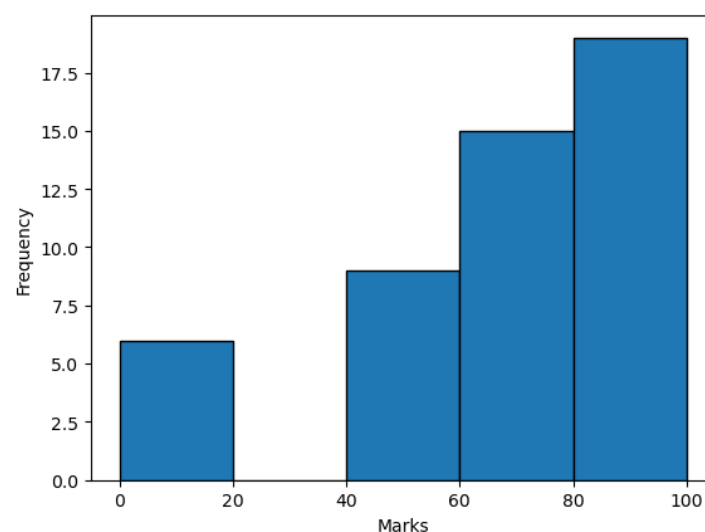
3. Histograms

A histogram shows the **distribution** of a dataset. It is especially useful for visualizing the frequency of numerical data within specified ranges, called **bins**.

Basic Histogram

```
1 import matplotlib.pyplot as plt
2
3 data = [96, 82, 75, 91, 67, 88, 59, 3, 84, 62,
4         90, 77, 68, 94, 56, 7, 7, 91, 70, 85,
5         92, 66, 74, 0, 97, 94, 60, 76, 83,
6         93, 78, 71, 58, 86, 72, 3, 69, 65, 98,
7         57, 99, 55, 81, 53, 52, 54, 1, 50, 100]
```

```
1 plt.hist(data, bins=5, edgecolor='black')
2 plt.xlabel("Marks")
3 plt.ylabel("Frequency")
```



Insight:

Paper was easy – most marks cluster toward the higher bins, with only a small tail of very low scores.

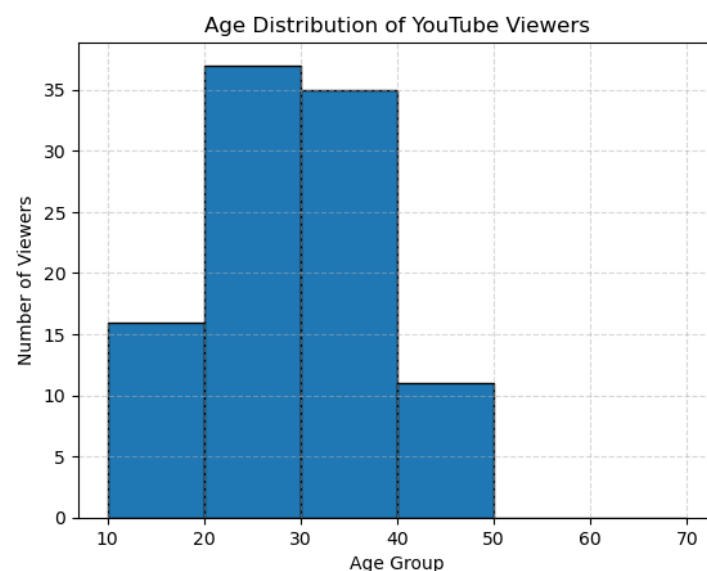
Custom Bin Edges

```
1 import matplotlib.pyplot as plt
2
3 ages = [
```

```

4     34, 28, 36, 45, 27, 27, 45, 37, 25, 35,
5     25, 25, 32, 10, 12, 24, 19, 33, 20, 15,
6     44, 27, 30, 15, 24, 31, 18, 33, 23, 27,
7     23, 48, 29, 19, 38, 17, 32, 10, 16, 31,
8     37, 31, 28, 26, 15, 22, 25, 40, 33, 12,
9     33, 26, 23, 36, 40, 39, 21, 26, 33, 39,
10    25, 28, 18, 18, 38, 43, 29, 40, 33, 23,
11    33, 45, 29, 45, 3, 38, 30, 27, 30, 10,
12    27, 33, 44, 24, 21, 24, 39, 33, 24, 35,
13    30, 39, 22, 26, 26, 15, 32, 32, 30, 27
14 ]
15
16 bins = [10, 20, 30, 40, 50, 60, 70]    # explicitly specifying bin edges
17
18 plt.hist(ages, bins=bins, edgecolor='black')
19 plt.title('Age Distribution of YouTube Viewers')
20 plt.xlabel('Age Group')
21 plt.ylabel('Number of Viewers')
22 plt.grid(True, linestyle='--', alpha=0.5)
23 plt.show()

```

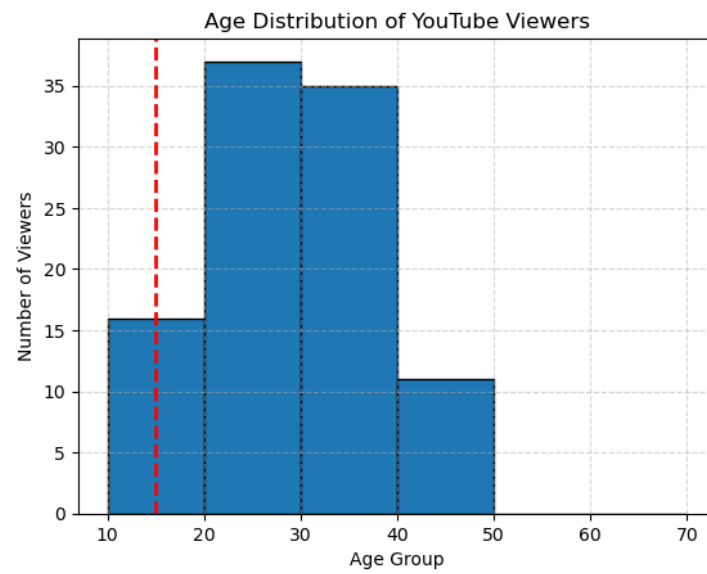


Adding a Vertical Reference Line: axvline

```

1 plt.hist(ages, bins=bins, edgecolor='black')
2 plt.axvline(15, color="red", linestyle="--", linewidth=2, label="
  Average")
3 plt.title('Age Distribution of YouTube Viewers')
4 plt.xlabel('Age Group')
5 plt.ylabel('Number of Viewers')
6 plt.grid(True, linestyle='--', alpha=0.5)
7 plt.show()

```

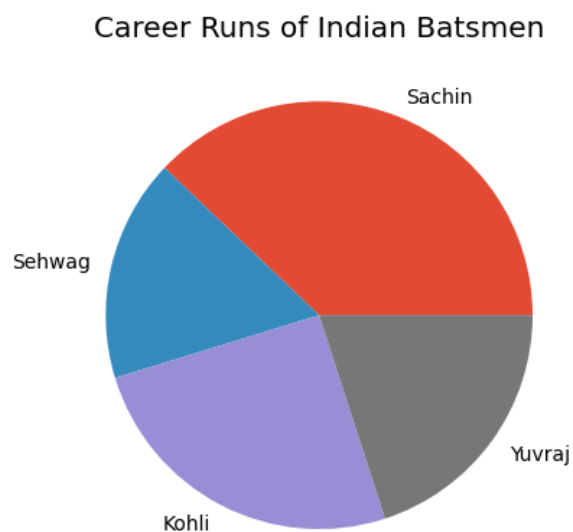


4. Pie Charts

Pie charts are mainly used when there are **fewer categories** – they represent each category as a proportional slice of a whole circle.

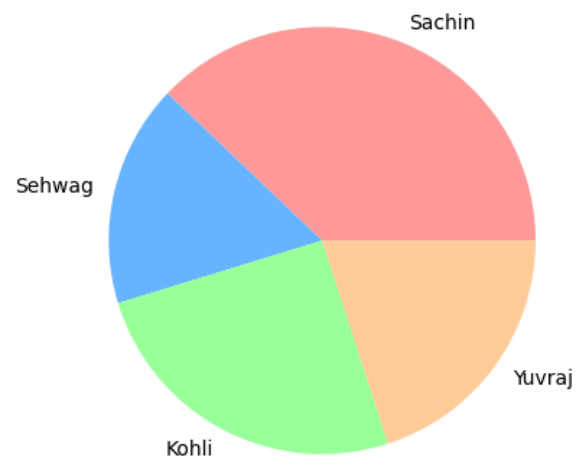
Basic Pie Chart

```
1 import matplotlib.pyplot as plt
2 plt.style.use("ggplot")    # choose any style you like
3
4 labels = ["Sachin", "Sehwag", "Kohli", "Yuvraj"]
5 runs = [18000, 8000, 12000, 9500]
6
7 plt.title("Career Runs of Indian Batsmen")
8 plt.pie(runs, labels=labels)
9 plt.show()
```



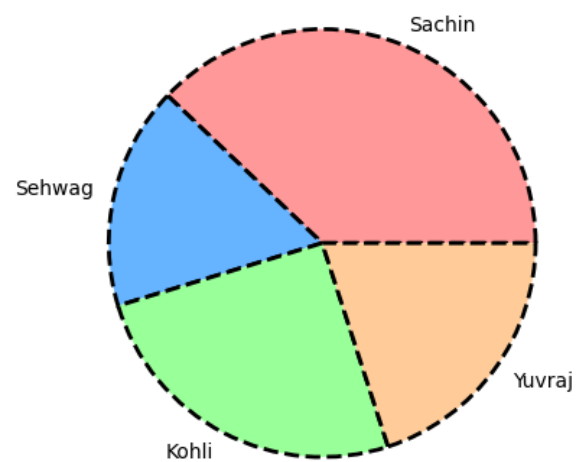
Custom Colors

```
1 colors = ['#ff9999', '#66b3ff', '#99ff99', '#ffcc99']
2 plt.pie(runs, labels=labels, colors=colors)
3 plt.show()
```



Wedge Properties

```
1 plt.pie(  
2     runs,  
3     labels=labels,  
4     colors=colors,  
5     wedgeprops={'edgecolor': 'black', 'linewidth': 2, 'linestyle': '--'}  
6 )  
7 plt.show()
```

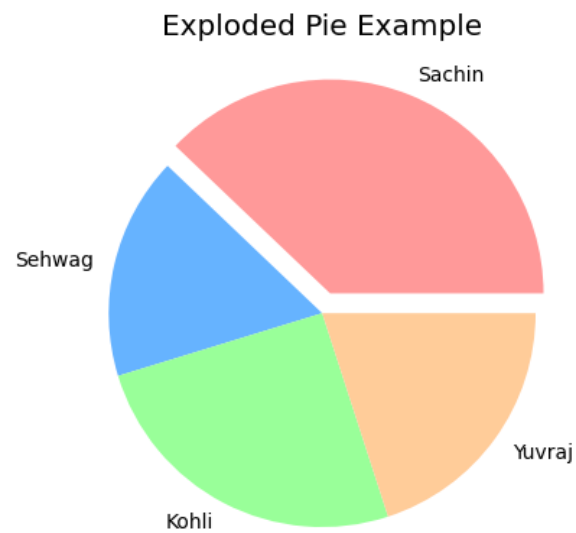


Tip:

Search “matplotlib wedgeprops” for the full list of slice-edge customization options.

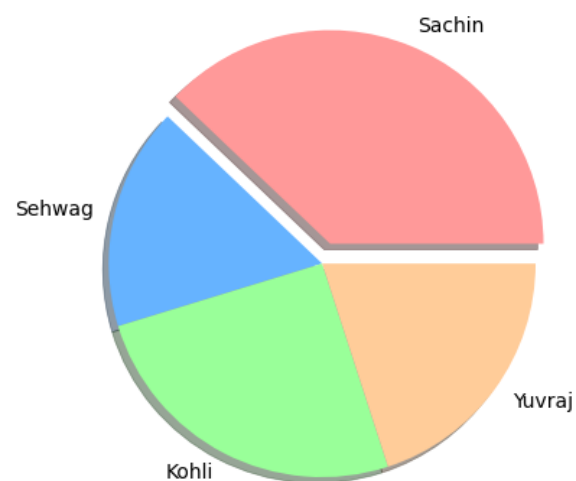
Exploded Slices

```
1 explode = [0.1, 0, 0, 0]    # Only highlight Sachin's slice
2
3 plt.pie(runs, labels=labels, explode=explode, colors=colors)
4 plt.title("Exploded Pie Example")
5 plt.show()
```



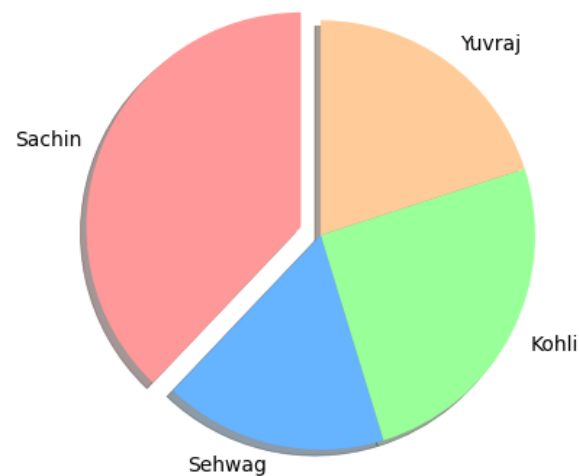
Shadow Effect

```
1 plt.pie(
2     runs,
3     labels=labels,
4     explode=explode,
5     colors=colors,
6     shadow=True    # adds a 3D-like shadow
7 )
8 plt.show()
```



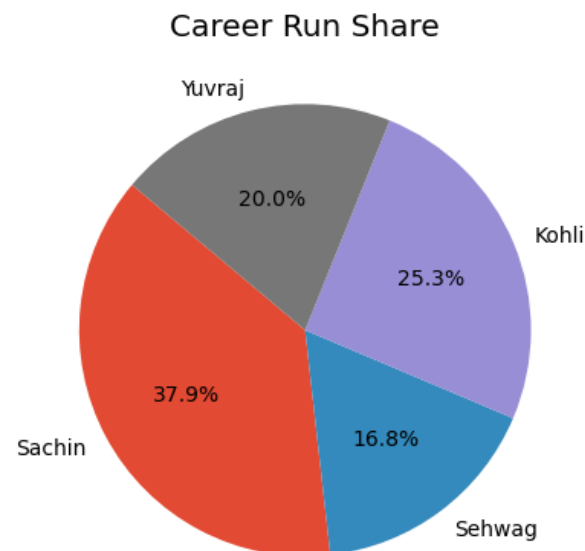
Start Angle

```
1 plt.pie(  
2     runs,  
3     labels=labels,  
4     explode=explode,  
5     colors=colors,  
6     shadow=True,  
7     startangle=90    # rotate the pie chart as desired  
8 )  
9 plt.show()
```



Percentage Labels: autopct

```
1 plt.pie(  
2     runs,  
3     labels=labels,  
4     autopct='%1.1f%%',    # floating point with precision 1  
5     startangle=140  
6 )  
7 plt.title("Career Run Share")  
8 plt.show()
```



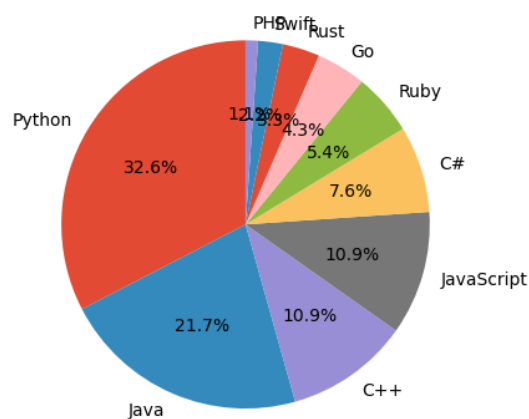
When NOT to Use a Pie Chart

Caution:

Too many categories make pie charts hard to read – avoid pie charts beyond 5–6 slices.

```
1 languages = ['Python', 'Java', 'C++', 'JavaScript', 'C#',  
2             'Ruby', 'Go', 'Rust', 'Swift', 'PHP']  
3 usage = [30, 20, 10, 10, 7, 5, 4, 3, 2, 1]  
4  
5 plt.pie(usage, labels=languages, autopct='%1.1f%%', startangle=90)  
6 plt.title("Programming Language Usage (Crowded Example)")  
7 plt.show()
```

Programming Language Usage (Crowded Example)



Summary of Useful Pie Chart Parameters

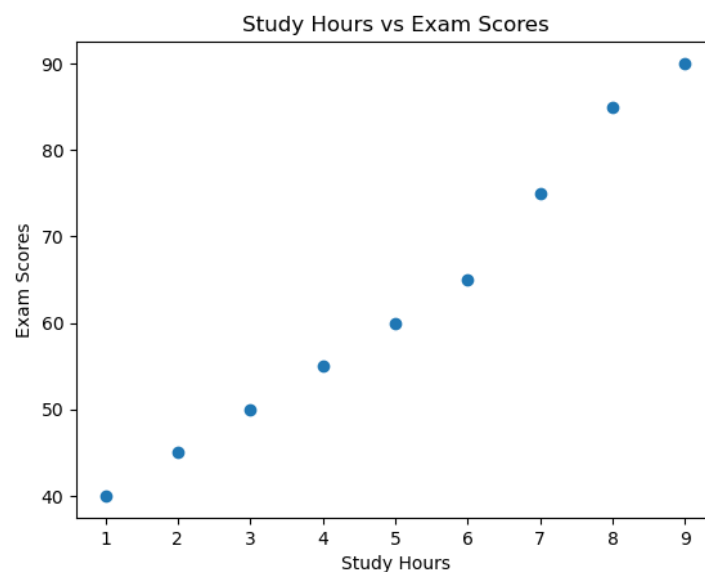
Parameter	Use
<code>labels</code>	Label each slice
<code>colors</code>	Customize slice colors (names or hex)
<code>explode</code>	Pull out slices for emphasis
<code>shadow</code>	Adds depth-like shadow
<code>startangle</code>	Rotates pie to start from a different angle
<code>autopct</code>	Shows percentage text on slices
<code>wedgeprops</code>	Customize slice edge, fill, width, etc.

5. Scatter Plots

Scatter plots are used to show the **relationship between two variables**. Here we plot study hours vs. exam scores for a group of students.

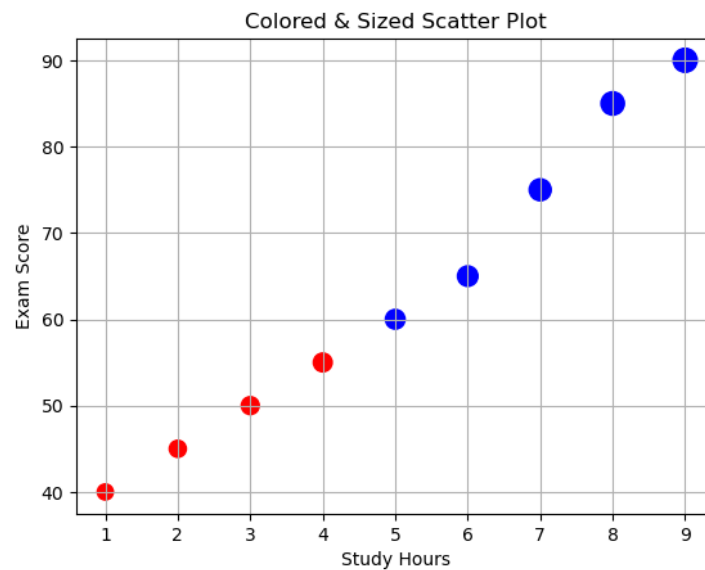
Basic Scatter Plot

```
1 import matplotlib.pyplot as plt
2
3 study_hours = [1,2,3,4,5,6,7,8,9]
4 exam_scores = [40, 45, 50, 55, 60, 65, 75, 85, 90]
5
6 plt.scatter(study_hours, exam_scores)
7 plt.title("Study Hours vs Exam Scores")
8 plt.xlabel("Study Hours")
9 plt.ylabel("Exam Scores")
```



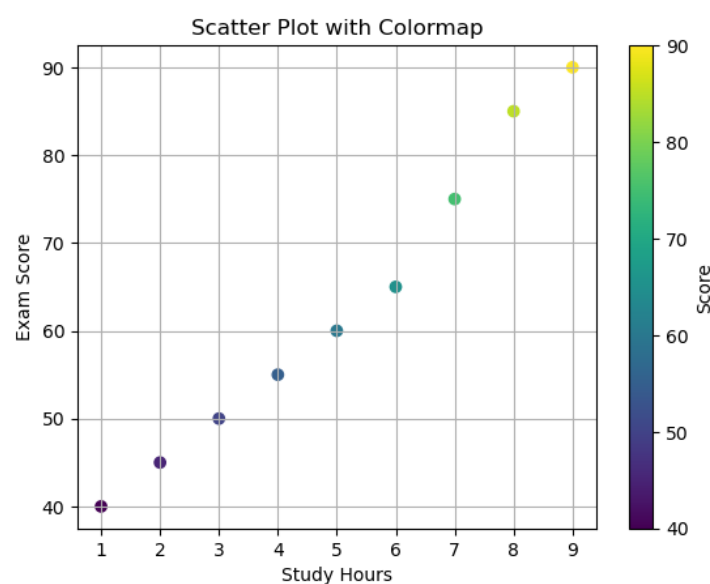
Size and Color Encoding

```
1 # Size of points based on score (bigger score -> bigger point)
2 sizes = [score * 2 for score in exam_scores]
3 colors = ['red' if score < 60 else 'blue' for score in exam_scores]
4
5 plt.scatter(study_hours, exam_scores, s=sizes, c=colors)
6 plt.title('Colored & Sized Scatter Plot')
7 plt.xlabel('Study Hours')
8 plt.ylabel('Exam Score')
9 plt.grid(True)
10 plt.show()
```



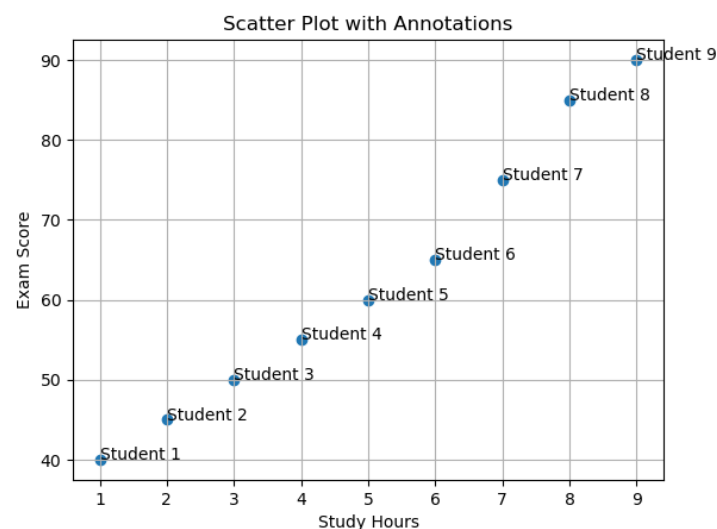
Colormaps with NumPy Arrays

```
1 import numpy as np
2 # Also works with NumPy arrays
3 scores_normalized = np.array(exam_scores)
4
5 plt.scatter(study_hours, exam_scores, c=scores_normalized, cmap='
6     viridis')
7 plt.colorbar(label='Score')
8 plt.title('Scatter Plot with Colormap')
9 plt.xlabel('Study Hours')
10 plt.ylabel('Exam Score')
11 plt.grid(True)
12 plt.show()
```



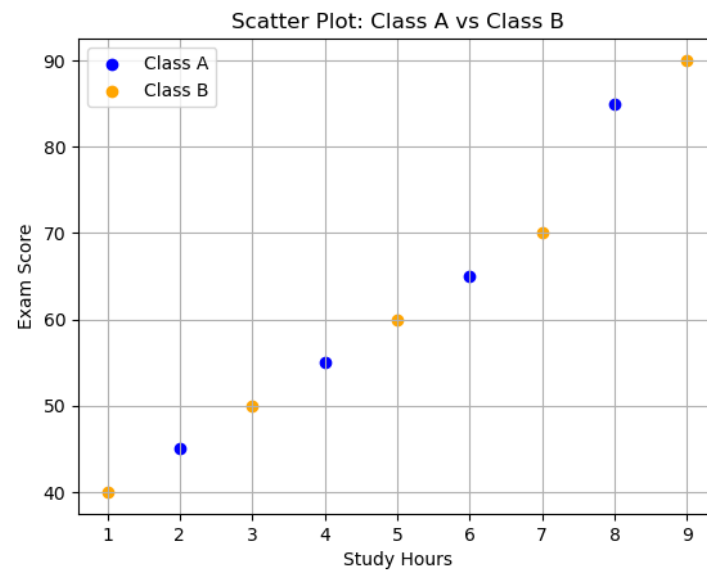
Point Annotations

```
1 plt.scatter(study_hours, exam_scores)
2
3 # Add labels
4 for i in range(len(study_hours)):
5     plt.annotate(f'Student {i+1}', (study_hours[i], exam_scores[i]))
6
7 plt.title('Scatter Plot with Annotations')
8 plt.xlabel('Study Hours')
9 plt.ylabel('Exam Score')
10 plt.grid(True)
11 plt.show()
```



Comparing Two Groups

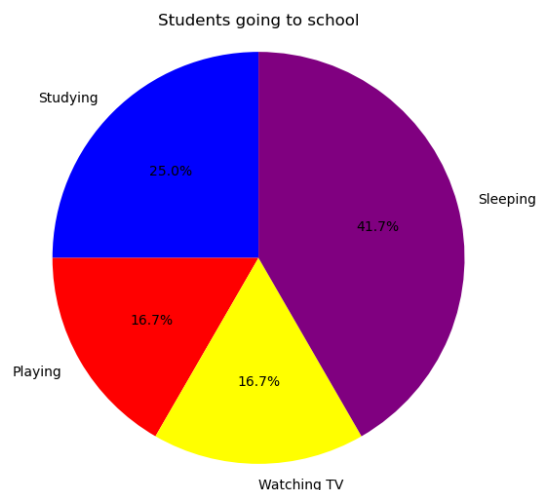
```
1 # Assume two groups: Class A and Class B
2 class_a_hours = [2, 4, 6, 8]
3 class_a_scores = [45, 55, 65, 85]
4
5 class_b_hours = [1, 3, 5, 7, 9]
6 class_b_scores = [40, 50, 60, 70, 90]
7
8 plt.scatter(class_a_hours, class_a_scores, label='Class A', color='blue')
9 plt.scatter(class_b_hours, class_b_scores, label='Class B', color='orange')
10
11 plt.title('Scatter Plot: Class A vs Class B')
12 plt.xlabel('Study Hours')
13 plt.ylabel('Exam Score')
14 plt.legend()
15 plt.grid(True)
16 plt.show()
```



6. Stack Plots

Recap: Pie Chart for a Single Snapshot

```
1 import matplotlib.pyplot as plt
2
3 activities = ["Studying", "Playing", "Watching TV", "Sleeping"]
4 time_spent = [3, 2, 2, 5]
5 colors = ['blue', 'red', 'yellow', 'purple']
6
7 plt.figure(figsize=(6,6)) # in inches
8 plt.pie(time_spent, labels=activities, colors=colors,
9         startangle=90, autopct="%1.1f%%")
10 plt.title("Students going to school")
11 plt.axis("equal") # makes the pie a perfect circle
```



The Limitation:

A pie chart only shows a single snapshot in time. To track how time spent on each activity changes *across days*, we need a **Stack Plot**.

What is a Stack Plot?

A Stack Plot is a type of area chart that visualizes multiple quantities over time, stacked on top of each other. It is especially useful to see how individual parts contribute to a whole over time.

Use Cases:

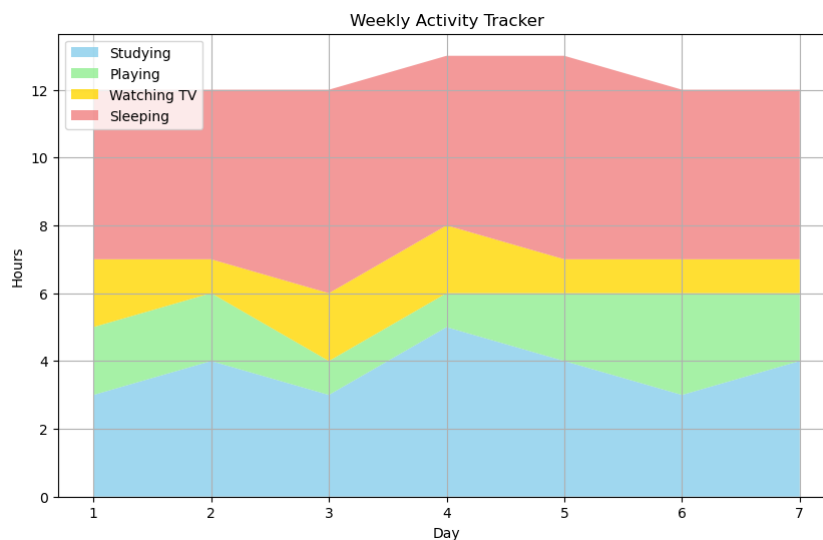
- Time spent on different activities over days
- Distribution of tasks by team members over a project timeline
- Website traffic sources over a week

Weekly Activity Tracker

```

1 import matplotlib.pyplot as plt
2
3 days = [1, 2, 3, 4, 5, 6, 7]    # Days of the week
4 studying = [3, 4, 3, 5, 4, 3, 4]
5 playing = [2, 2, 1, 1, 2, 3, 2]
6 watching_tv = [2, 1, 2, 2, 1, 1, 1]
7 sleeping = [5, 5, 6, 5, 6, 5, 5]
8
9 labels = ['Studying', 'Playing', 'Watching TV', 'Sleeping']
10 colors = ['skyblue', 'lightgreen', 'gold', 'lightcoral']
11
12 plt.figure(figsize=(10,6))
13 plt.stackplot(days, studying, playing, watching_tv, sleeping,
14               labels=labels, colors=colors, alpha=0.8)
15
16 plt.legend(loc='upper left')    # Location of the legend
17 plt.title('Weekly Activity Tracker')
18 plt.xlabel('Day')
19 plt.ylabel('Hours')
20 plt.grid(True)
21 plt.show()

```



Key Parameters in `stackplot()`

Parameter	Description
<code>x</code>	The x-axis data (like days)
<code>*args</code>	Multiple y-values (like studying, playing, etc.)
<code>labels</code>	Labels for the legend
<code>colors</code>	List of colors for each stack
<code>alpha</code>	Transparency level (0 to 1)
<code>loc</code> (in legend)	Position of the legend ('upper left', 'best', etc.)

Summary:

- Use pie charts for a snapshot in time.
- Use stack plots to see how data changes over time while still showing parts of a whole.
- Customize your plot using parameters like `colors`, `labels`, `alpha`, and `legend`.
- Stack plots are a great way to tell a story over time.

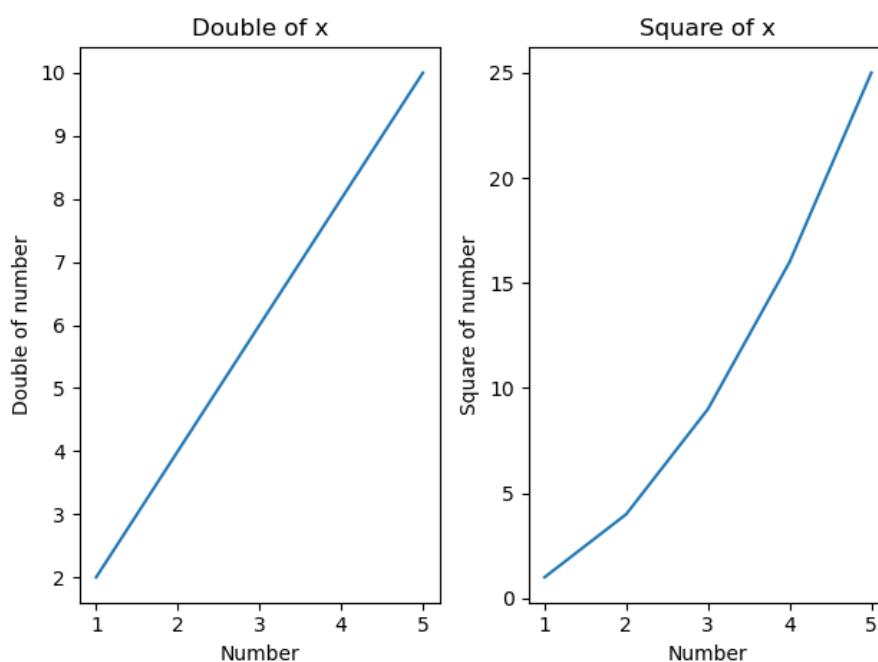
7. Subplots

Subplots allow you to show multiple plots in a single figure, side-by-side or in a grid layout. This is helpful when comparing different datasets or aspects of the same data.

```
1 import matplotlib.pyplot as plt
2
3 # Data
4 x = [1, 2, 3, 4, 5]
5 y1 = [i * 2 for i in x]
6 y2 = [i ** 2 for i in x]
```

`plt.subplot(): Row, Column, Index`

```
1 plt.subplot(1,2,1)    # row, column, which plot
2 plt.plot(x,y1)
3 plt.title("Double of x")
4 plt.xlabel("Number")
5 plt.ylabel("Double of number")
6
7 plt.subplot(1,2,2)    # row, column, which plot
8 plt.plot(x,y2)
9 plt.title("Square of x")
10 plt.xlabel("Number")
11 plt.ylabel("Square of number")
12 plt.tight_layout()
```

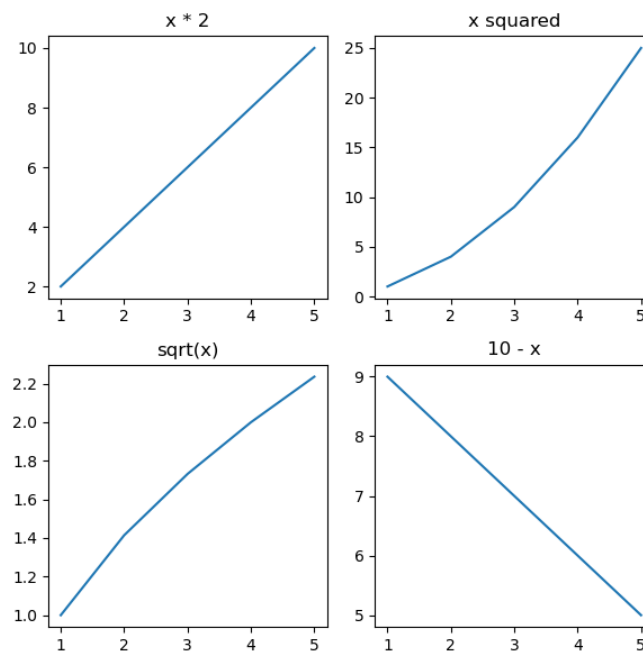


A 2×2 Grid of Plots

```

1 # More variations of x
2 y3 = [i ** 0.5 for i in x]
3 y4 = [10 - i for i in x]
4
5 plt.figure(figsize=(6, 6))    # Optional: make it bigger
6
7 plt.subplot(2, 2, 1)
8 plt.plot(x, y1)
9 plt.title('x * 2')
10
11 plt.subplot(2, 2, 2)
12 plt.plot(x, y2)
13 plt.title('x squared')
14
15 plt.subplot(2, 2, 3)
16 plt.plot(x, y3)
17 plt.title('sqrt(x)')
18
19 plt.subplot(2, 2, 4)
20 plt.plot(x, y4)
21 plt.title('10 - x')
22
23 plt.tight_layout()
24 plt.show()

```



`plt.subplots()`: The Object-Oriented Way

```

1 fig, axs = plt.subplots(1, 2, figsize=(10, 4))
2
3 axs[0].plot(x, y1)
4 axs[0].set_title('x * 2')

```

```

5
6  axs[1].plot(x, y2)
7  axs[1].set_title('x squared')
8
9  fig.suptitle('Simple Comparison Plots', fontsize=14)
10 fig.tight_layout()
11 fig.subplots_adjust(top=0.85)    # So title doesn't overlap
12 fig.savefig('my_plots.png')      # Save as image
13
14 plt.show()

```

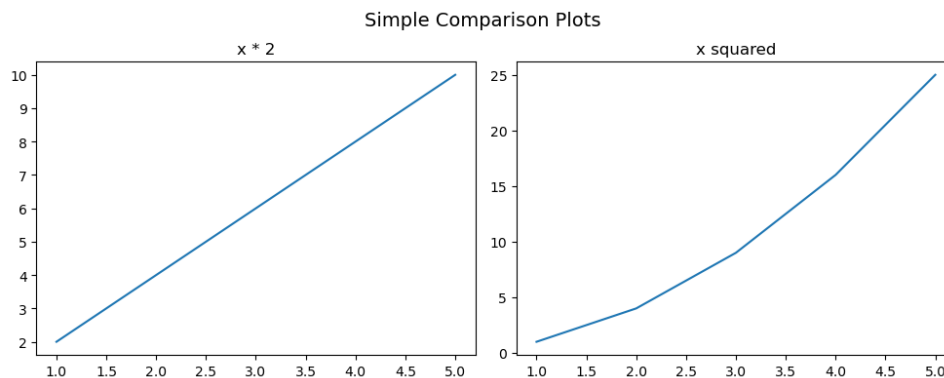


Figure vs. Axes:

Figure = the whole sheet of paper.

Subplot (Axes) = individual graphs drawn on that sheet.

`fig, axs = plt.subplots(1, 2, figsize=(10, 4))` breaks down as:

- `plt.subplots(rows, columns)` – here (1, 2) means 1 row, 2 columns $\Rightarrow$ 2 graphs side by side.
- `fig` stores the entire figure (the whole window).
- `axs` stores all the subplots as a list: `axs = [graph1, graph2]`, so `axs[0]` is the left graph and `axs[1]` is the right graph.
- `figsize=(10,4)` controls the size in inches – width $\times$ height. A larger width makes the plots wider.

A 2 $\times$ 2 Grid with the OO Interface

```

1  fig, axs = plt.subplots(2, 2, figsize=(8, 6))
2  ys = [y1, y2, y3, y4]
3  titles = ['x * 2', 'x squared', 'sqrt(x)', '10 - x']
4
5  for i in range(2):
6      for j in range(2):
7          idx = i * 2 + j
8          axs[i, j].plot(x, ys[idx])
9          axs[i, j].set_title(titles[idx])
10
11 plt.tight_layout()
12 plt.show()

```

